BÀI TẬP TÌM KIẾM TRONG AI

Họ và tên	Tô Gia Huy
MSSV / Lớp	3123410131 / DCT1232

Mục tiêu học tập

• Mô phỏng các thuật toán BFS, DFS và Iterative Deepening.

• Biểu diễn bài toán Tháp Hà Nội dưới dạng bài toán tìm kiếm.

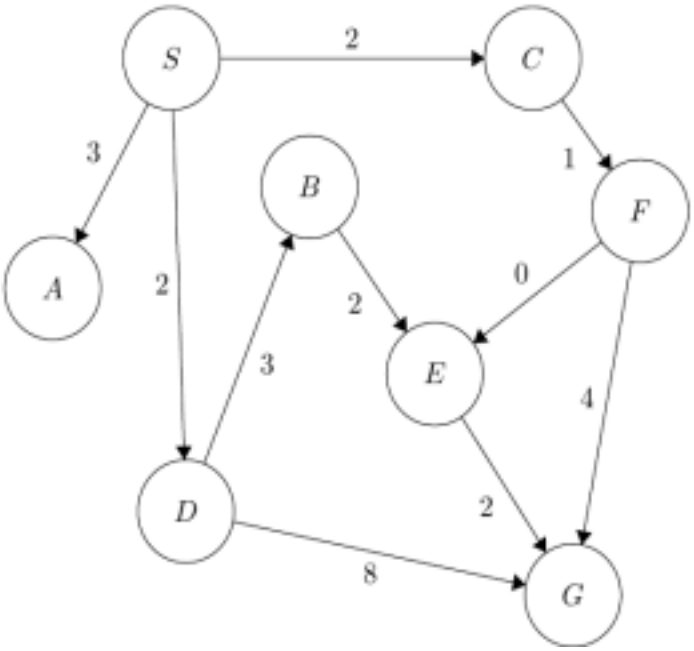
• Thiết kế và phân tích heuristic cho A*.

• So sánh DFS, BFS, UCS, Greedy Search và A* trên cùng một bài toán.

• Phân tích các phát biểu lý thuyết về heuristic, A*, Minimax và Expectimax.

Bài 1. Các thuật toán tìm kiếm

Cho đồ thị có trạng thái bắt đầu S và trạng thái đích G như hình dưới đây.



Hình 1. Đồ thị dùng cho các thuật toán tìm kiếm

Yêu cầu: Với mỗi thuật toán, hãy ghi thứ tự các nút được đưa vào explored set, đường đi được trả về và tổng chi phí của đường đi. Nếu có hòa, phá hòa theo thứ tự alphabet của trạng thái cuối cùng trong đường đi.

1.1. Breadth-First Search (BFS)

Nội dung	Kết quả
Explored Set	S, A, C, D, F, B
Path	S → D → G

Total Cost	2 + 8 = 10

Giải quyết vấn đề bằng tìm kiếm **2** / **8**

1.2. Depth-First Search (DFS)

Nội dung	Kết quả
Explored Set	S, A, C, F, E
Path	S → C → F → E → G
Total Cost	2 + 1 + 0 + 2 = 5

1.3. Iterative Deepening Search (IDS)

Giới hạn độ sâu	Các nút được duyệt
0	S
1	S, A, C, D
2	S, A, C, F, D, B, G
...	

Path cuối cùng:

$S \rightarrow D \rightarrow G$

Total Cost: 2+8=10

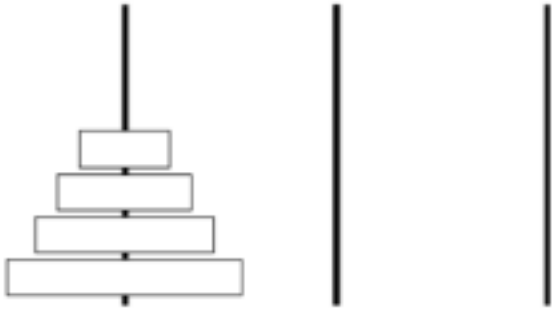
Câu hỏi thảo luận: BFS và DFS khác nhau ở cấu trúc frontier và tính tối ưu như thế nào?

Tiêu chí	**BFS**	**DFS**
Cấu trúc Frontier	Queue (FIFO)	Stack (LIFO) hoặc đệ quy
Cách duyệt	Theo chiều rộng, ưu tiên nút nông nhất	Đi sâu nhất có thể rồi quay lui

Tính đầy đủ	Có, nếu hệ số phân nhánh hữu hạn	Không đảm bảo với không gian tìm kiếm vô hạn
Tính tối ưu	Có nếu các cạnh có chi phí bằng nhau	Không đảm bảo
Bộ nhớ	Thường tốn nhiều bộ nhớ	Thường ít tốn bộ nhớ hơn

Bài 2. Tháp Hà Nội - Tower of Hanoi

Bài toán gồm n đĩa có kích thước khác nhau và ba cọc. Ban đầu các đĩa được xếp trên một cọc theo thứ tự từ lớn đến nhỏ. Mỗi lần chỉ được di chuyển một đĩa và không được đặt đĩa lớn lên trên đĩa nhỏ. Mục tiêu là chuyển toàn bộ đĩa sang cọc thứ ba.



Hình 2. Trạng thái ban đầu của bài toán Tháp Hà Nội

2.1. Biểu diễn trạng thái

Hãy đề xuất cách biểu diễn một trạng thái của bài toán. Biểu diễn phải cho biết mỗi đĩa đang ở cọc nào và đủ để xác định các hành động hợp lệ.

Giả sử có n đĩa được đánh số từ 1 đến n (trong đó 1 là đĩa nhỏ nhất, n là đĩa lớn nhất) và 3 cọc được đánh số là $1, 2, 3$. Một trạng thái S có thể được biểu diễn bằng một tuple gồm 3 danh sách (hoặc ngăn xếp), đại diện cho 3 cọc: $S = (C_1, C_2, C_3)$. Mỗi danh sách chứa các số nguyên tương ứng với các đĩa đang nằm trên cọc đó, được sắp xếp theo thứ tự từ dưới lên trên. Vì luật của bài toán là đĩa nhỏ phải nằm trên đĩa lớn, các số trong mỗi danh sách luôn giảm dần từ trái sang phải (hoặc từ dưới lên trên đỉnh). _ **2.2. Kích thước không gian trạng thái**

Với n đĩa, hãy xác định số trạng thái có thể có theo cách biểu diễn của bạn và giải thích cách suy luận.

Kích thước không gian trạng thái là 3^n.

Giải thích suy luận:

- Có n đĩa, mỗi đĩa có thể nằm ở một trong 3 cọc bất kỳ.
- Số cách phân bố n đĩa vào 3 cọc mà không quan tâm thứ tự là $3 \times 3 \times \ldots \times 3 = 3^n$.
- Tuy nhiên, do ràng buộc của bài toán là **không được đặt đĩa lớn lên trên đĩa nhỏ**, nên một khi đã xác định được tập hợp các đĩa nào nằm trên một cọc cụ thể, thứ tự sắp xếp của chúng trên cọc đó là duy nhất (bắt buộc phải xếp từ lớn nhất ở dưới cùng đến nhỏ nhất ở trên cùng).
- Do đó, mọi cách gán vị trí cọc cho n đĩa đều tương ứng với đúng một trạng thái hợp lệ. Tổng số trạng thái có thể có là 3^n.

_

Giải quyết vấn đề bằng tìm kiếm **3** / **8**

_

_ 2.3. Trạng thái bắt đầu

Hãy viết trạng thái khởi đầu S0 theo cách biểu diễn đã chọn.

Trạng thái khởi đầu S_0 là khi toàn bộ n đĩa đều nằm trên cọc đầu tiên theo thứ tự từ lớn đến nhỏ, và hai cọc còn lại rỗng: $S_0 = ([n, n-1, \ldots, 2, 1], [], [])$

_ 2.4. Các hành động hợp lệ

Từ một trạng thái bất kỳ, hãy mô tả tập Actions(s). Một hành động hợp lệ phải thỏa mãn các điều kiện nào?

Từ một trạng thái S bất kỳ, tập $Actions(S)$ bao gồm các hành động có dạng $Move(i, j)$ - nghĩa là "di chuyển đĩa nằm trên cùng của cọc i sang đặt lên trên cùng của cọc j" (với $i, j \in \{1, 2, 3\}$ và $i \neq j$).

Một hành động $Move(i, j)$ là hợp lệ khi thỏa mãn đồng thời các điều kiện sau:

1. **Cọc nguồn i không được rỗng** (phải có ít nhất một đĩa để di chuyển).
2. **Cọc đích j rỗng**, HOẶC nếu cọc j không rỗng thì **đĩa trên cùng của cọc i phải nhỏ hơn đĩa trên cùng của cọc j** (đảm bảo không vi phạm luật đĩa lớn đè lên đĩa nhỏ).

_

_ 2.5. Goal Test

Hãy mô tả hàm goalTest(state): khi nào hàm trả về True?

Mục tiêu là chuyển toàn bộ đĩa sang cọc thứ ba. Hàm $goalTest(state)$ trả về True khi trạng thái hiện tại S đạt được mục tiêu, tức là cọc 1 và cọc 2 rỗng, cọc 3 chứa tất cả các đĩa:

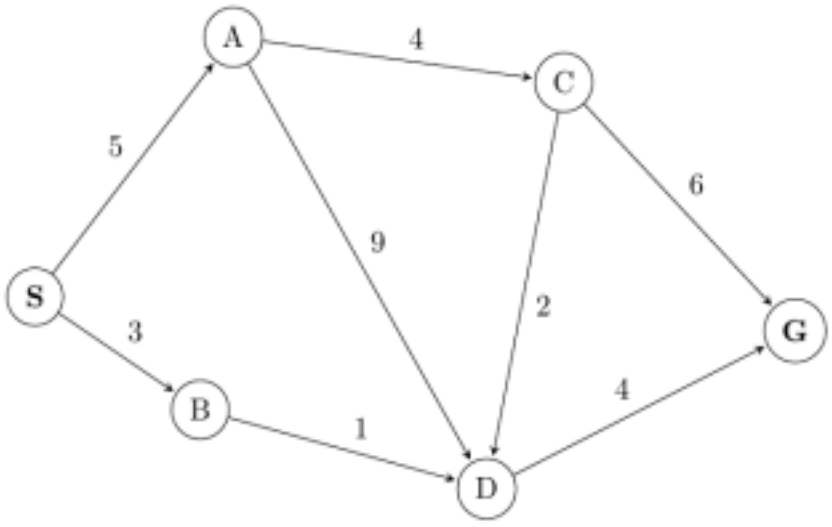
- Điều kiện trả về True: $S = ([], [], [n, n-1, \ldots, 2, 1])$

_

Bài 3. Thiết kế và phân tích Heuristic

Xét đồ thị dưới đây. Hãy thiết kế heuristic và phân tích ảnh hưởng của heuristic đến thuật toán

A*.



Hình 3. Đồ thị dùng để thiết kế heuristic

Công thức A*: f(n) = g(n) + h(n), trong đó g(n) là chi phí thực tế từ S đến n và h(n) là chi phí ước lượng từ n đến đích.

3.1. Thiết kế heuristic admissible và consistent

Điền giá trị h(n) cho từng nút sao cho heuristic phù hợp với yêu cầu của nhóm.

Node	h(n)
S	8
A	10
B	5
C	6
D	4
G	0

3.2. Chạy A* với heuristic đã thiết kế

Explored Set:

S, B, D, G

Path: S → B → D → G

Total Cost: 8

3.3. Heuristic admissible nhưng không consistent

Đề xuất một bộ giá trị h(n) admissible nhưng không consistent. Chỉ ra ít nhất một cạnh vi phạm điều kiện h(n) <= c(n,n') + h(n').

Để một heuristic không consistent, ta cần tạo ra ít nhất một cạnh vi phạm điều kiện $h(n) \leq c(n, n') + h(n')$. Tuy nhiên, nó vẫn phải admissible, tức là $h(n) \leq h^*(n)$ với mọi n. Ta đề xuất bộ giá trị $h(n)$ sau:

- $h(S) = 8$
- $h(A) = 10$
- $h(B) = 5$ (thỏa mãn $\leq h^*(B) = 5$)
- $h(C) = 6$
- $h(D) = 1$ (thỏa mãn $\leq h^*(D) = 4$)
- $h(G) = 0$

Chỉ ra cạnh vi phạm: Xét cạnh $B \rightarrow D$ có chi phí thực tế $c(B, D) = 1$. Theo điều kiện consistent: $h(B)$ phải $\leq c(B, D) + h(D)$. Thế số vào ta có: $5 \leq 1 + 1$, tương đương $5 \leq 2$ (Điều này là **Sai**). Vậy cạnh $B \rightarrow D$ đã vi phạm tính consistent, trong khi toàn bộ các giá trị $h(n)$ đều không vượt quá chi phí thực tế đến đích (vẫn đảm bảo admissible).

_ 3.4. Câu hỏi mở rộng

Giải thích vì sao một heuristic consistent thì cũng admissible, giả sử h(G) = 0.

_____

_

_____

_

_____

_

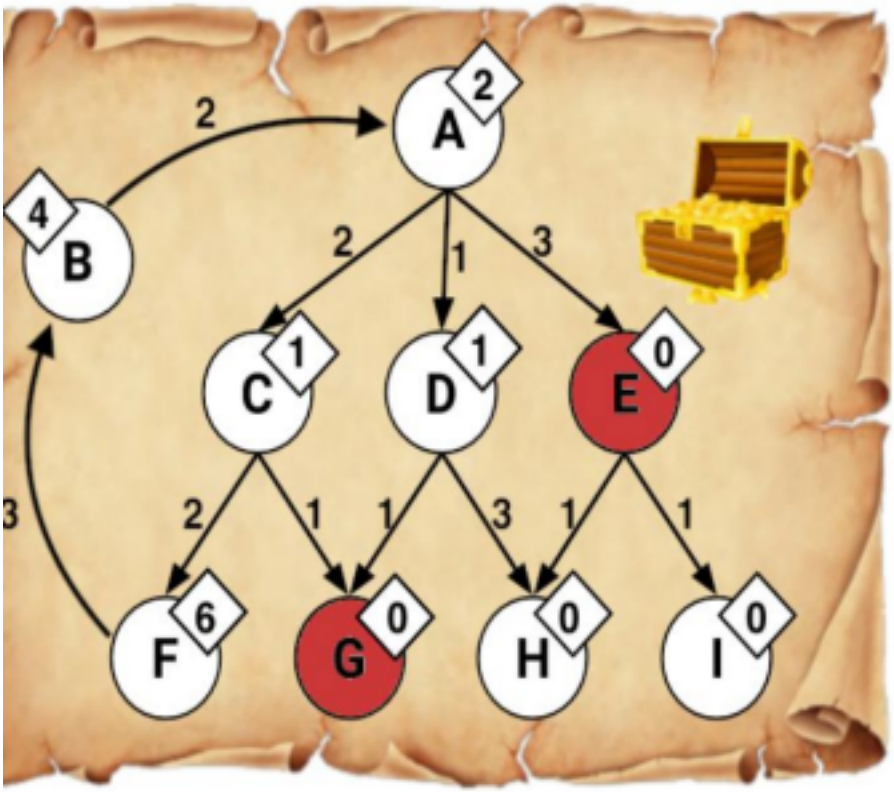
_____

_

Bài 4. Treasure Hunting (Tìm kho báu)

A là đảo xuất phát. Các nút màu đỏ là các đảo có kho báu; chỉ cần tới một trong các đảo đó. Mũi tên biểu diễn hành động có hướng; số trên cạnh là action cost. Giá trị trong hình thoi là heuristic ước lượng chi phí tối thiểu từ nút hiện tại tới một kho báu.

Giải quyết vấn đề bằng tìm kiếm **5** / **8**



Hình 4. Bản đồ tìm kho báu (giữ nguyên hình gốc)

Quy tắc phá hòa: khi lấy phần tử khỏi frontier, nếu hòa thì chọn theo thứ tự alphabet.

4.1. Depth-First Search

Thuật toán ưu tiên duyệt theo chiều sâu, xét theo thứ tự alphabet.

- Bắt đầu tại A. Các nút kề là B, C, D, E. Đưa vào frontier. Rút B ra trước.
- Từ B, không có đường đi tiếp tới nút mới chưa duyệt. Lùi lại.
- Rút C ra xét. Từ C đi đến F, G. Đưa F, G vào frontier. Rút F ra trước.
- Từ F đi đến B (đã duyệt). Lùi lại.

- Rút G ra xét. G là nút đích. Dừng thuật toán.

Explored Set:

A, B, C, F, G

_ Path returned:

A → C → G _

_ 4.2. Breadth-First Search

thuật toán duyệt theo từng mức độ sâu, ưu tiên thứ tự alphabet.

- Bắt đầu tại A. Đưa B, C, D, E vào frontier (queue).
- Lấy B ra xét.
- Lấy C ra xét (đưa F, G vào frontier).
- Lấy D ra xét (đưa H vào frontier, G đã có).
- Lấy E ra xét. E là nút đích. Dừng thuật toán.
- Explored Set: A, B, C, D, E
- Path returned: A → E

Explored Set:

A, B, C, D, E

_ Path returned:

A → E

4.3. Uniform-Cost Search

Thuật toán ưu tiên chọn nút có chi phí thực tế $g(n)$ nhỏ nhất.

- Bắt đầu: $A(g = 0)$. Lấy A ra. Frontier: $D(1), B(2), C(2), E(3)$.
- Lấy $D(1)$ ra xét. Frontier cập nhật: $B(2), C(2), G(1 + 1 = 2), E(3), H(1 + 3 = 4)$.
- Lấy $B(2)$ ra xét (không có kề mới).
- Lấy $C(2)$ ra xét (đường $C \rightarrow G$ tốn $2 + 1 = 3$, nhưng G đã có trong frontier với $g = 2$ nên giữ nguyên).
- Lấy $G(2)$ ra xét. G là đích. Dừng.

Explored Set:

A, D, B, C, G

_ Path returned:

A → D → G

_ **Cost:** 2

_ **4.4. Greedy Search**

Thuật toán chỉ quan tâm đến chi phí ước lượng $h(n)$.

- Bắt đầu: $A(h = 2)$. Lấy A ra. Cập nhật frontier gồm các nút kề kèm h: $E(0), C(1), D(1), B(4)$.
- Lấy nút có h nhỏ nhất là $E(0)$. E là đích. Dừng.

Explored Set:

A, E

_ **Path returned:**

A → E

_ **Cost:** 3

4.5. A* Search

thuật toán chọn nút dựa trên $f(n) = g(n) + h(n)$.

- Bắt đầu: $A(f = 0 + 2 = 2)$. Lấy A ra. Frontier có: $D(g = 1, h = 1 \rightarrow f = 2)$, $C(g = 2, h = 1 \rightarrow f = 3)$, $E(g = 3, h = 0 \rightarrow f = 3)$, $B(g = 2, h = 4 \rightarrow f = 6)$.
- Lấy $D(f = 2)$ ra xét (nhỏ nhất). Mở rộng ra G: $g(G) = 1 + 1 = 2, h(G) = 0 \rightarrow f(G) = 2$. Mở rộng ra H: $g = 1 + 3 = 4, h = 0 \rightarrow f = 4$. Frontier cập nhật: $G(f = 2), C(f = 3), E(f = 3), H(f = 4), B(f = 6)$.
- Lấy $G(f = 2)$ ra xét. G là đích. Dừng.

Explored Set:

A, D, G

_ **Path returned:**

A → D → G

_ **Cost:** 2

Bảng tổng hợp so sánh

Thuật toán	Tiêu chí chọn node	Path	Cost
DFS	Độ sâu	A → C → G	3

Giải quyết vấn đề bằng tìm kiếm **7** / **8**

BFS	Độ sâu nhỏ nhất	A → E	3
UCS	g(n)	A → D → G	2

Greedy	h(n)	A → E	3
A*	g(n)+h(n)	A → D → G	2

Bài 5. Câu hỏi Đúng/Sai

Với mỗi phát biểu, hãy ghi Đúng hoặc Sai và giải thích ngắn gọn. Nếu sai, nên đưa ra phản ví dụ.

Câu 1. Depth-first search luôn mở rộng ít nhất nhiều node bằng A* sử dụng một admissible heuristic. Đúng / Sai: sai

Giải thích: DFS có thể tình cờ đi đúng ngay vào nhánh chứa trạng thái đích và tìm thấy đích rất nhanh (chỉ bằng đúng độ sâu của đích), trong khi A* có thể phải mở rộng thêm nhiều node ở các nhánh khác để đảm bảo tính tối ưu của đường đi.

Câu 2. Giả sử một quân xe trong cờ vua có thể đi bất kỳ số ô theo hàng hoặc cột trong một nước đi và không thể nhảy qua quân khác. Manhattan distance là một admissible heuristic cho số nước đi nhỏ nhất từ ô A đến ô B. Đúng / Sai: sai

Giải thích: Khoảng cách Manhattan tính bằng tổng số ô theo chiều ngang và dọc. Tuy nhiên, quân xe có thể đi nhiều ô trên cùng một hàng/cột chỉ trong 1 bước di chuyển. Ví dụ: từ (0,0) đến (0,4) mất 1 bước, nhưng khoảng cách Manhattan là 4. Vì $h(n) = 4 > h^*(n) = 1$, heuristic này đánh giá quá cao chi phí thực tế nên không admissible.

Câu 3. Euclidean distance là một admissible heuristic cho bài toán tìm đường của Pacman. Đúng / Sai: đúng

Giải thích: Khoảng cách Euclidean là đường thẳng nối trực tiếp hai điểm. Đường thẳng luôn là khoảng cách ngắn nhất. Pacman di chuyển trên lưới nên chi phí thực tế (số ô phải đi) luôn lớn hơn hoặc bằng khoảng cách đường thẳng. Do đó $h(n) \leq h^*(n)$ luôn đúng.

Câu 4. Tổng của nhiều admissible heuristics vẫn luôn là một admissible heuristic.

Đúng / Sai: sai

Giải thích: Nếu hai heuristic $h_1(n)$ và $h_2(n)$ đều ước lượng rất sát hoặc bằng chi phí thực tế $h^*(n)$, thì tổng $h_1(n) + h_2(n)$ có thể sẽ lớn hơn $h^*(n)$, vi phạm điều kiện admissible (không bao giờ được đánh giá cao hơn thực tế). (Thông thường ta dùng hàm $\max(h_1, h_2)$ để giữ tính admissible).

Câu 5. Admissibility của một heuristic cho A* kéo theo consistency.

Đúng / Sai: sai

Giải thích: Mệnh đề ngược lại mới đúng (Consistency kéo theo Admissibility). Một heuristic có thể admissible (luôn nhỏ hơn chi phí thật) nhưng lại không consistent nếu giá trị đánh giá giảm quá nhanh giữa một node cha và node con, vi phạm bất đẳng thức tam giác $h(n) \leq c(n, n') + h(n')$

Câu 6. A* với graph search luôn tối ưu nếu heuristic admissible.

Đúng / Sai: sai

Giải thích: Để thuật toán A* đảm bảo tính tối ưu khi chạy trên **graph search** (tìm kiếm đồ thị - có lưu lại các node đã xét trong Explored set), heuristic phải đạt điều kiện **consistent** (nhất quán). Nếu chỉ admissible, thuật toán có thể bỏ qua một đường đi tốt hơn đến một node đã được đưa vào Explored set trước đó.

Tổng kết nhanh

Nhóm	Thuật toán	Hàm/tiêu chí chính
Uninformed Search	BFS / DFS / IDS	Không sử dụng heuristic
Cost-based Search	UCS	g(n)
Informed Search	Greedy	h(n)

Giải quyết vấn đề bằng tìm kiếm **8** / **8**

Informed Search	A*	g(n)+h(n)

Ghi nhớ: A* dùng f(n) = g(n) + h(n).

---o0o---

(Hết)